

Choreographing Safety: Planning via Ice-cone-Inspired Motion Sets of Feedback Controllers for Car-like Robots

Veejay Karthik J¹, Anushka Verma², Leena Vachhani³

Abstract—Safe navigation of car-like robots operating via onboard sensing on crowded roads is a challenging task in general. Our primary objective in this work is the development of a novel sensor-based control structure that directly computes safe control inputs for navigation based on the motion sets of pre-designed feedback controllers. We demonstrate an input-constrained feedback controller to induce motion sets whose geometric structure is an ice-cone. The key implication of this is that the naturally induced system trajectories evolve within this motion set during stabilization. We then propose an instantaneous planning strategy for safe navigation that choreographs a sequence of ice-cones within the bounds of safety. This enables the robot to navigate safely without the explicit requirement of adhering to pre-computed paths/trajectories. We demonstrate the proposed methodology via ROS simulations on test scenarios using the CAT vehicle testbed. Further, we demonstrate its application in a traffic intersection scenario using the trajectory information collected from the interaction dataset. The proposed approach especially finds applications in scenarios where pre-computing safe trajectories based on the predicted trajectories of neighboring agents is difficult/unreliable during run-time.

Index Terms—feedback control, car-like robots, safe control, collision avoidance, sensor-based

I. INTRODUCTION

Wheeled mobile robots find applications in a wide spectrum of domains ranging from indoor warehouses, museums, and elderly care to interplanetary explorations, road driving, and agriculture. As they are often deployed in constrained operating environments, collision avoidance during navigation is a fundamental aspect to be dealt with. For example, warehouses are controlled environments with organized layouts, predefined motion paths, relatively static scenarios, and predictable motion of dynamic entities. In contrast, the scenarios such as crowded roads, the presence of unpredictable elements such as pedestrians, cyclists, motorists, etc, demand fast reactivity for safe navigation. More often than not, anticipating their actions can be challenging. Operating in controlled environments with relatively static scenarios allows for efficient planning, while driving in uncontrolled crowded environments demands higher levels

of adaptability and fast reactivity to handle the dynamic and unpredictable nature of the surroundings to ensure safety. The conventional approaches for safe navigation follow a plan-control sequence. In such approaches, safe open-loop (reference) trajectories are first planned based on anticipated trajectories of the neighboring agents in the robot's vicinity. A feedback controller is then post-designed to track the planned trajectories for navigation.

Velocity obstacles (VO) and their variants [1] are designed for enabling safe motion planning in dynamic multi-agent scenarios. Further, techniques such as optimal reciprocal collision avoidance (ORCA) [2] built over the VO concept provide a more collaborative and decentralized collision avoidance strategy. However, the knowledge of the precise velocities and positions of the neighboring agents is a prerequisite. Moreover, instantaneous generation of safe velocities for navigation assumes a shape of the robot as circular or polytope [3].

With the advent of sophisticated optimization tools, optimization-based methods [4], [5] for real-time trajectory planning of car-like robots have been designed to safely operate in constrained dynamic environments. Model predictive control (MPC) schemes are state-of-the-art optimization-based feedback controllers capable of encoding the system dynamics and their associated constraints (on both states and inputs) in their online, finite horizon optimization problem for generating feasible control inputs. The MPC schemes are typically suited for autonomous driving scenarios where, control inputs are generated for tracking the planned trajectory [6], [7]. The planning horizons are practically limited by the available onboard computational resources for control computations to maneuver safely through the dynamically varying reference trajectories generated based on the collision-free regions ahead of the robot. Further, directly synthesized safe control inputs for navigation have been obtained by enforcing set invariance as state-dependant constraints via control barrier functions [8], [9]. Recent MPC variants like tinyMPC [10] (considers linear system dynamics for control computations) have been tailored for embedded platforms. However, for non-linear systems such as car-like robots, non-linear MPC formulations, which are computationally demanding in general, are often required for real-time applications to compute control inputs reliably under unmeasured velocities of dynamic obstacles/co-existing agents.

In particular, the ensuring safe navigation in environments with co-existing agents/robots such as crowded roads needs to address following challenges:

This work is supported through the Prime Minister's Research Fellowship (PMRF ID: 1302088)

¹Veejay Karthik J is a doctoral candidate at ARMS Lab, Systems and Control Engineering, Indian Institute of Technology Bombay, Mumbai, India veejay@iitb.ac.in

²Anushka Verma is a sophomore in the department of Aerospace Engineering, Indian Institute of Technology Bombay, Mumbai, India. anushkav1888@gmail.com

³Leena Vachhani is a professor in the department of Systems and Control Engineering, Indian Institute of Technology Bombay, Mumbai, India. leena.vachhani@iitb.ac.in

- Predicting the motion of the dynamic agents during run-time becomes very difficult from raw sensor data during run-time (can potentially induce delays, affecting reactivity). Consequently, planning safe trajectories in the presence of multiple dynamic entities becomes challenging.
- A significant number of re-planning cycles would be required due to the open-loop nature of trajectory planning. This becomes critical in resource-constrained platforms as predicting the trajectories of the neighboring agents and planning based on the same often require significant resources.

In recent times, deep reinforcement learning (DRL)-based methods have emerged as promising techniques for enabling intelligent navigation in both static [11] and dynamic [12] cluttered environments. The DRL methods are especially attractive for robot navigation tasks as they are capable of generating control policies that directly map raw sensor data into control inputs for navigation. As safety is a non-negotiable aspect of autonomous driving applications, a multitude of safe RL frameworks [13], [14] have been proposed for low-level decision-making. However, the machine learning methods are still unable to guarantee autonomous driving safety in complex environments, especially for multiple dynamic obstacles with undetermined velocities [15].

Unlike existing work, we investigate an integrated control structure for navigation that directly computes guaranteed safe control inputs based on the point-cloud data from onboard sensing elements during run-time. We propose a problem formulation that establishes an integrated control structure directly mapping to the combination of point-cloud data and the pre-designed feedback controller. The pre-designed controller through associated ice-cone motion sets synthesizes safe control inputs for navigation. The attempt is to avoid explicit requirement of adhering to planned paths/trajectories.

The key contributions and their organization in this work are as follows: Section II describes the novel problem setup and the associated mathematical models involved. The proposed input-constrained feedback controller and its capability to constrain the position trajectories within an ice-cone motion set during stabilization is described in section III. Section IV proposes characterization of the set of admissible reference points for the feedback controller based on the proposed ice-cone motion sets, and a strategy for selecting reference points to synthesize safe control inputs for navigation. Finally, MATLAB simulations are carried out to demonstrate the proposed navigator in an interesting traffic intersection scenario (from the interaction dataset) in section V. Also, ROS simulations using CAT vehicle testbed are carried out for demonstrating safe navigation in parallel and perpendicular crossing scenarios in this section. The results show safe navigation in the presence of multiple dynamic entities and crossing/head-on scenarios without explicitly designing collision-free paths.

II. PROBLEM FORMULATION

A. System Description

The ego-vehicle is a car-like robot modeled through the front wheel-driven bicycle model (illustrated in Figure 1) in this work. Throughout this article, the superscript ‘ k ’ represents the time instance. The associated kinematics are as follows,

$$\begin{bmatrix} \dot{x}^k \\ \dot{y}^k \\ \dot{\theta}^k \\ \dot{\phi}^k \end{bmatrix} = \begin{bmatrix} \cos(\theta^k) \cos(\phi^k) \\ \sin(\theta^k) \sin(\phi^k) \\ \frac{\sin(\phi^k)}{L} \\ 0 \end{bmatrix} v_1 + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} v_2$$

The control inputs (v_1, v_2) represent the front-wheel velocity and the steering rate, respectively. As illustrated in Figure 1, the rear wheel position is at (x^k, y^k) , the ego-vehicle’s orientation is θ^k , and the steering angle is ϕ^k . The distance between the front and rear wheel axles is given by L . The tread of the ego-vehicle is T . To simplify the control design procedure, the following transformations [16] are carried out,

- An alternate set of generalized co-ordinates $(x_f^k, y_f^k, \delta^k, \theta^k)$ is defined through the following state transformations,

$$\begin{aligned} x_f^k &= x^k + L \cos(\theta^k) ; & y_f^k &= y^k + L \sin(\theta^k) \\ \delta^k &= \theta^k + \phi^k \end{aligned}$$

Here, (x_f^k, y_f^k) represents the front wheel coordinates, and δ^k is the absolute steering angle relative to x -axis.

- The transformed control inputs (ω_1, ω_2) are obtained through the following bijective transformation,

$$\begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{1}{L} \sin(\delta^k - \theta^k) & 1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} ; \quad \delta^k = \theta^k + \phi^k, \quad (1)$$

where (ω_1, ω_2) represent the front wheel velocity and the rate of absolute steering angle (δ) respectively.

Consequently, the transformed system kinematics becomes,

$$\begin{bmatrix} \dot{x}_f^k \\ \dot{y}_f^k \\ \dot{\delta}^k \\ \dot{\theta}^k \end{bmatrix} = \begin{bmatrix} \cos(\delta^k) \\ \sin(\delta^k) \\ 0 \\ \frac{\sin(\delta^k - \theta^k)}{L} \end{bmatrix} \omega_1 + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \omega_2 \quad (2)$$

Remark 1: The front and rear wheel axles are connected via a rigid body link, so they are always ‘ L ’ units apart.

At the k^{th} time instance, the ego-vehicle’s states are defined with respect to a reference point $\mathcal{W}^k$ as illustrated in Figure 1. The state $\vec{R}^k$ is the radial position vector, and the state ψ^k is the relative bearing with respect to the position vector originating from $\mathcal{W}^k$ and terminating on the front wheel axle’s midpoint ($\mathcal{R}$). The magnitude of the radial position vector is described through $\|\vec{R}^k\|$. In this context, the engagement kinematics of the system is described as,

$$\begin{bmatrix} \|\dot{\vec{R}}^k\| \\ \dot{\psi}^k \end{bmatrix} = \begin{bmatrix} \omega_1 \cos(\psi^k) \\ \omega_2 - \frac{\omega_1}{\|\vec{R}^k\|} \sin(\psi^k) \end{bmatrix} \quad (3)$$

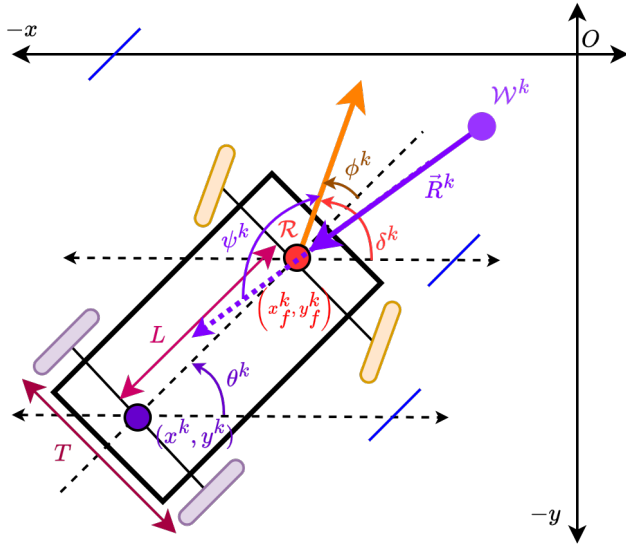


Fig. 1: System Description (Ego-Vehicle) - Car-like robot modeled through front wheel driven bicycle model

The stabilization of the kinematics in (3) ensures that $\mathcal{R}$ stabilizes at the reference point $\mathcal{W}^k$ relative to which $(\vec{R}^k, \psi^k)$ is described. Consequently, the feedback controller designed for accomplishing this ensures that the midpoint of the rear-wheel axle (x^k, y^k) reaches a ball of radius L around $\mathcal{W}^k$ (rigid body constraint).

B. Problem Description

Given that the dynamic obstacles/agents in the crowd intend to navigate safely, the ego-vehicle has point-cloud information on the obstacles in its vicinity. This information is obtained via an onboard LiDAR with a limited field of view mounted on the midpoint of the front wheel axle. On invoking a feedback controller for stabilization, the set in which the naturally induced system trajectories evolve is a function of the references provided to the controller, and the nature of the feedback control design itself. In this context, the problems addressed in this work are as follows:

- Design of an input-constrained feedback controller for a car-like robot modeled through a front wheel-driven bicycle model. The feedback controller naturally induces position trajectories such that they are constrained within a motion set shaped as an ice-cone during stabilization.
- Design of a strategy to generate references such that the associated motion sets (ice-cone) are constrained within the space constraints imposed by the point-cloud data obtained during run-time. This ensures the control inputs that are synthesized are safe for navigation.

The proposed strategy to generated references is presented in section IV, while the associated input-constrained feedback controller is developed in the next section.

III. FEEDBACK CONTROL DESIGN

For the engagement kinematics in (3), the stabilizing feedback controller $\mathcal{F}(\vec{R}^k, \psi^k; \mathcal{W}^k)$ [17] is designed in terms of the feedback states $(\vec{R}^k, \psi^k)$ described relative to $\mathcal{W}^k$. For the sake of brevity, the arguments are dropped, and it is represented as $\mathcal{F}^k$ in the rest of this work.

$$\begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix} = \begin{bmatrix} -K_1 \tanh(\|\vec{R}^k\|) \operatorname{sgn}(c(\psi^k)) \\ -K_2 |\sigma^k|^{\frac{1}{2}} \operatorname{sgn}(\sigma^k) \\ -K_1 \left(\frac{\tanh(\|\vec{R}^k\|)}{\|\vec{R}^k\|} \right) \operatorname{sgn}(c(\psi^k)) s(\psi^k) \end{bmatrix} \quad (4)$$

Here, $c(\cdot) = \cos(\cdot)$, $s(\cdot) = \sin(\cdot)$, $\operatorname{sgn}(\cdot)$ is the signum function, $\tanh(\cdot)$ is the hyperbolic tangent function, and $K_1 > 0, K_2 > 0$ are the control gains (design choices).

$$\operatorname{sgn}(x) = \begin{cases} 1 & x > 0 \\ \{-1 \quad 1\} & x = 0 \\ -1 & x < 0 \end{cases} \quad (5)$$

The manifold σ^k based on the relative bearing $(\psi^k(0))$ at the instant $(t = 0)$ when it begins its navigation towards $\mathcal{W}^k$,

$$\sigma^k = \begin{cases} \psi^k - \operatorname{sgn}(\psi^k)\pi, & -1 \leq \cos(\psi^k(0)) < 0 \\ \psi^k, & 0 \leq \cos(\psi^k(0)) \leq 1 \end{cases} \quad (6)$$

The key features of the feedback controller $\mathcal{F}^k$ in (4) are characterized through the following lemmas,

Lemma 1: (Global Stability) The naturally induced position trajectories by $\mathcal{F}_i$ are globally stable, and attractive towards $\mathcal{W}^k$. They also exhibit asymptotic stability when $\|\vec{R}^k\| \rightarrow 0$ as $\tanh(\|\vec{R}^k\|) \sim \|\vec{R}^k\|$ then holds.

Proof: Refer Theorem 2 [17] ■

Lemma 2: (Euclidean distance to goal) The naturally induced position trajectories by $\mathcal{F}^k$ are such that their radial distances to $\mathcal{W}^k$ are always monotonically decreasing.

Proof: The Lyapunov function chosen to demonstrate position stability is the squared radial distance to the $\mathcal{W}^k$. The negative definite nature of the time derivative of this function guarantees that the radial distance is monotonically decreasing (Refer Theorem 2 [17]). ■

Lemma 3: (Finite-time alignment to $\mathcal{W}^k$) The orientation (δ) aligns (in either parallel/anti-parallel configuration) in the direction of the instantaneous position vector originating from $\mathcal{W}^k$ in finite-time when $\mathcal{F}^k$ is invoked. The consequent induced motion in any configuration is such that the robot always approaches $\mathcal{W}^k$.

Proof: Refer Lemma 2, Theorem 1 [17] ■

Lemma 4: (Persistent alignment to $\mathcal{W}^k$) The orientation (δ) remains persistently aligned (either in parallel/anti-parallel configuration) in the direction of the instantaneous position vector originating from $\mathcal{W}^k$ when $\mathcal{F}^k$ is invoked. In any configuration, the resulting induced motion is such that the robot always approaches $\mathcal{W}^k$.

Proof: The orientation stability achieved by enforcing ψ^k trajectories onto $\sigma^k = 0$ (from (6)) ensures that they

monotonically align (either in parallel/anti-parallel configuration) in the direction of the instantaneous position vector, thereby ensuring persistence in alignment. Refer Lemma 2, and Theorem 1 [17] for details. ■

Lemma 5: (Perpendicular alignment distance to $\mathcal{W}^k$) The perpendicular alignment distance $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$ to $\mathcal{W}^k$ monotonically decreases with time.

$$d_{\mathcal{W}^k}(\vec{R}^k, \delta^k) = \left| \begin{bmatrix} -\sin(\delta) \\ \cos(\delta) \end{bmatrix}^T \vec{R}^k \right| = \|\vec{R}^k\| \cdot |\sin(\psi^k)|$$

Proof: Trivially, the following proof should hold true when $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k) \neq 0 \implies \|\vec{R}^k\| \neq 0, \psi^k \neq \{0, \pi\}$. Therefore, the rate of change of $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$ is computed as follows,

$$\begin{aligned} \frac{d}{dt}(d_{\mathcal{W}^k}^2(\mathcal{R}, \delta)) &= \frac{d}{dt}(\|\vec{R}^k\|^2 \sin^2(\psi^k) + \|\vec{R}^k\|^2 \frac{d}{dt}(\sin^2(\psi^k))) \\ &= \underbrace{-2K_1 \|\vec{R}^k\| \tanh(\|\vec{R}^k\|) |c(\psi^k)| s^2(\psi^k)}_{T_1} \\ &\quad \underbrace{-2K_2 \|\vec{R}^k\| c(\psi^k) s(\psi^k) |\sigma^k|^{\frac{1}{2}} \text{sgn}(\sigma^k)}_{T_2} \end{aligned}$$

Only when $\psi^k = \{\frac{\pi}{2}, -\frac{\pi}{2}\}$, the following is true,

$$\frac{d}{dt}(d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)) = 0 \quad (7)$$

However, (7) occurs only when the initial conditions forces this case, and it never persists as $\psi^k = \{\frac{\pi}{2}, -\frac{\pi}{2}\}$ is not an equilibrium of the system (Theorem 1 [17]) under the feedback control action in (4).

The following arguments prove that the rate of change of $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$ is always negative in any other case.

- It is trivial that the term T_1 is always negative since each individual multiplicative term is always positive, and there is an overall multiplicative negative sign.
- To show that the term T_2 is also always negative, the following analysis is carried out.

$$\text{sgn}(\sigma^k) = \begin{cases} -\text{sgn}(s(\psi^k)), & \text{if } -1 \leq c(\psi^k) < 0 \\ \text{sgn}(s(\psi^k)), & \text{if } 0 < c(\psi^k) \leq 1 \end{cases} \quad (8)$$

Now, using (8) the following is true in relation to T_2 ,

$$T_2 = \begin{cases} 2K_2 \|\vec{R}^k\| \cdot c(\psi^k) \cdot |s(\psi^k)| \cdot |\sigma^k|^{\frac{1}{2}}, & \text{when } -1 \leq c(\psi^k) < 0 \\ -2K_2 \|\vec{R}^k\| \cdot c(\psi^k) \cdot |s(\psi^k)| \cdot |\sigma^k|^{\frac{1}{2}}, & \text{when } 0 < c(\psi^k) \leq 1 \end{cases} \quad (9)$$

Here, $c(\cdot) = \cos(\cdot)$, $s(\cdot) = \sin(\cdot)$, $\text{sgn}(\cdot)$ is the signum function, $\tanh(\cdot)$ is the hyperbolic tangent function

In both cases of (9), $T_2 < 0$ always. Therefore,

$$\begin{aligned} \frac{d}{dt}(d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)) &= \frac{1}{2d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)}(T_1 + T_2) \\ \implies \frac{d}{dt}(d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)) &< 0 \quad \forall \|\vec{R}^k\| \neq 0, \psi^k \neq \{0, \pi, \pm\frac{\pi}{2}\} \end{aligned}$$

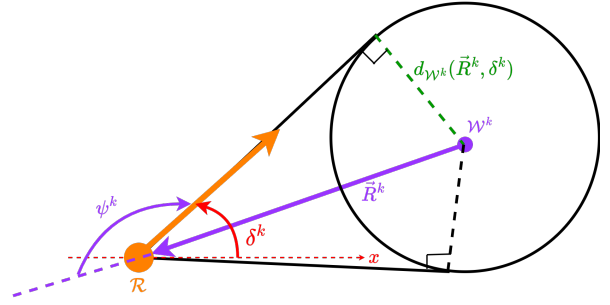


Fig. 2: Ice-Cone Motion Set. Its apex is at $\mathcal{R}$, base is at $\mathcal{W}^k$ and its radius is $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$. The orange arrow represents the absolute steering angle, and the purple arrow represents the position vector with respect to the reference point $\mathcal{W}^k$.

Since the time derivative of $d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$ is always negative, the lemma holds true. ■

Theorem 1: (Motion set invariance) The naturally induced position trajectories on invoking $\mathcal{F}^k$ evolve within a motion set whose geometric structure is an ice-cream cone $\mathcal{C}(a, b, \rho)$ while eventually converging onto $\mathcal{W}^k$.

$$\mathcal{C}(a, b, \rho) = \{a + \alpha(z - a) | \alpha \in [0, 1], z \in B(b, \rho)\}$$

Here, $B(b, \rho)$ is a ball centered at b with radius ρ . The ice-cone's apex and base are $a \in \mathbb{R}^2$ and $b \in \mathbb{R}^2$, respectively. The naturally induced position trajectories are such that $\mathcal{R}$ begins its motion from the apex 'a' and converges to the base 'b' of $\mathcal{C}(a, b, \rho)$.

(The parameters of $\mathcal{C}(a, b, \rho)$ can be visualized in Figure 2 as $a \equiv \mathcal{R}, b \equiv \mathcal{W}^k, \rho \equiv d_{\mathcal{W}^k}(\vec{R}^k, \psi^k)$)

Proof: From Propositions 4,5 [18], for a feedback controller to naturally induce trajectories such that they evolve with an ice-cone motion set during stabilization, it should exhibit the following characteristics. From Figure 2,

- The position trajectories (from $\mathcal{R}$) exhibit global stability and are attracted towards $\mathcal{W}^k$ (refer Lemma 1), and their radial euclidean distance to $\mathcal{W}^k$ keeps monotonically decreasing (refer Lemma 2).
- Finite-time and persistent alignment to $\mathcal{W}^k$ is achieved (refer Lemmas 3,4), and the perpendicular alignment distance ($d_{\mathcal{W}^k}(\vec{R}^k, \delta^k)$) to $\mathcal{W}^k$ keeps monotonically decreasing (refer Lemma 5).

Therefore, since the proposed $\mathcal{F}(\vec{R}^k, \psi^k; \mathcal{W}^k)$ possesses the requisite characteristics (via Lemmas referenced as above), it naturally induces trajectories that evolve within an ice-cone motion set during stabilization. Refer Lemmas 1,2 for stabilization to $\mathcal{W}^k$. ■

Theorem 2: (Bounded Controls) The control inputs (v_1, v_2) generated by $\mathcal{F}^k$ are bounded, and their bounds are as follows,

$$|v_1| \leq K_1 \quad ; \quad |v_2| \leq K_2 \frac{\pi}{2} + K_1 \left(1 + \frac{1}{L}\right)$$

Proof: From the input transformation (1),

$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{\sin(\delta - \theta)}{L} & 1 \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_2 \end{bmatrix} \quad (10)$$

The transformed inputs (ω_1, ω_2) generated by $\mathcal{F}^k$ are bounded as follows [17],

$$|\omega_1| \leq K_1 \quad ; \quad |\omega_2| \leq K_2 \frac{\pi}{2} + K_1 \quad (11)$$

- Its trivial from (10) that, since $v_1 = \omega_1$,

$$|v_1| \leq K_1$$

- From (10),(11)

$$\begin{aligned} v_2 &= \omega_2 - \omega_1 \frac{\sin(\delta - \theta)}{L} \\ \Rightarrow |v_2| &\leq K_2 \frac{\pi}{2} + K_1 \left(1 + \frac{1}{L}\right) \quad \because |\sin(\delta - \theta)| \leq 1 \end{aligned}$$

IV. SAFE CONTROL SYNTHESIS

A. Describing & Quantifying the set of admissible $\mathcal{W}^k$

As the ego-vehicle navigates through its environment, the local geometry of the operating workspace is perceived through the onboard LiDAR. The range information is obtained as a discrete set of tuples $\{(d_i, \theta_i)\}$ for $i \in \{0, \dots, N-1\}$, where N is the number of range measurements (illustrated as red dots in Figure 3). Considering a limited field-of-view, the discrete set of angles Θ^d at which the 2D point cloud data is obtained is within the set Θ measured relative to δ ,

$$\Theta^d \subset \Theta := [-\pi + \theta_{\lim}, \pi - \theta_{\lim}]$$

Here, $\theta_{\lim}$ represents the angle limit parameter.

Remark 2: It is to be noted here that if $\theta_{\lim} = 0$, then the LiDAR from which the point-cloud data is obtained would be a 360° LiDAR. Further, to account for the ego-vehicle's dimensions, the measured $d_i \rightarrow (d_i - \frac{T}{2})$ for computations (' T ' is the tread of the ego-vehicle illustrated in Figure 1). Reference points $\mathcal{W}^k$ are selected such that the associated motion set (ice-cone) satisfies the space constraints imposed by $\{(d_i, \theta_i)\}$. To mathematically characterize the set of such candidate points - $\mathcal{W}^k$, the angular space around $\mathcal{R}$ is discretized in coherence with discrete angular intervals at which the range data points are obtained.

For an ice-cone whose apex is at $\mathcal{R}$, and base at $\mathcal{W}^k$ (at a distance d in the relative direction of θ_i from $\mathcal{R}$), its boundary points (yellow markers on top of the ice-cone in Figure 3) must satisfy the following inequalities for it to not violate the space constraints imposed by $\{(d_i, \theta_i)\}$.

$$d \leq \frac{d_j}{\cos(\theta_j - \theta_i) + \sqrt{\sin^2(\theta_i) - \sin^2(\theta_j - \theta_i)}} \quad \forall \theta_j \in \mathcal{S}(\theta_i) \quad (12)$$

Each constraint in (12) is illustrated via yellow markers on top of the ice-cone in Figure 3. Here, the set $\mathcal{S}(\theta_i)$ is described as follows,

$$\mathcal{S}(\theta_i) = \begin{cases} \{\theta_j \in \Theta^d : \underbrace{0 \leq \theta_j \leq 2\theta_i}_{E_1}\} & \text{if } \frac{\pi - \theta_{\lim}}{2} \geq \theta_i \geq 0 \\ \{\theta_j \in \Theta^d : \underbrace{2\theta_i \leq \theta_j \leq 0}_{E_2}\}, & \text{if } \frac{-\pi + \theta_{\lim}}{2} \leq \theta_i \leq 0 \end{cases} \quad (13)$$

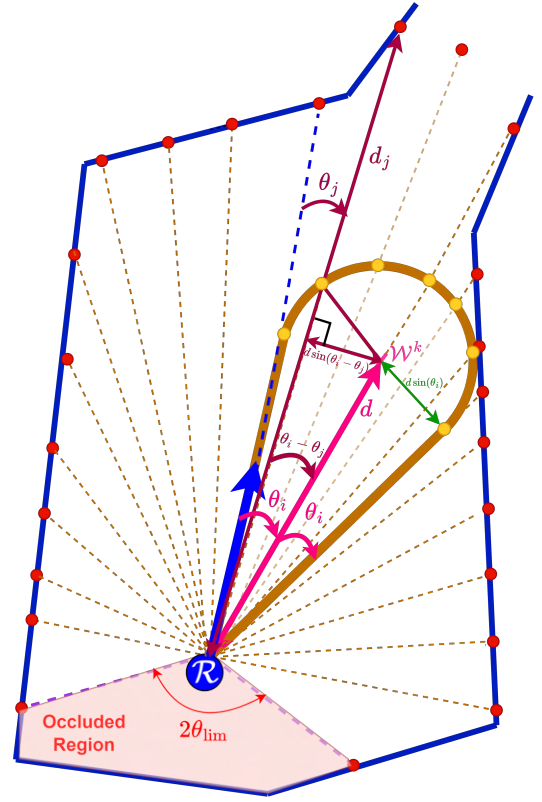


Fig. 3: An example scenario illustrating an ice-cone fit (brown boundary) within the space constraints imposed by $\{(d_i, \theta_i)\}$ (red markers). The blue arrow indicates the direction of the absolute steering angle δ of the ego-vehicle

Remark 3: Here, $\pi > \theta_{\lim} \geq \frac{\pi}{2}$ is considered as it is generally true for cars with front-fitted LiDARs.

To store the largest distance at which $\mathcal{W}^k$ can be selected in a relative direction θ_i , we describe the function $\mathcal{D} : \Theta^d \rightarrow \mathbb{R}^+$ as follows,

$$\mathcal{D}(\theta_i) = \max_d d, \quad \text{subject to (12)}$$

Remark 4: $\mathcal{D}(\theta_i)$ is an implicit function of δ as θ_i is measured relative to δ .

Therefore, the set of admissible reference points $\mathcal{A}(x_f, y_f, \delta) \subset \mathbb{R}^2$ around $\mathcal{R}$ is described as follows,

$$\mathcal{A} = \{(x_f + R \cos(\delta + \theta_i), y_f + R \sin(\delta + \theta_i)) : R \leq \mathcal{D}(\theta_i), \theta_i \in \Theta_S^d\} \quad (14)$$

$$\Theta_S^d = \{\theta_i \in \Theta^d : 2\theta_i \in \Theta\} \quad (15)$$

Θ_S^d is a subset of Θ^d such that, for any $\theta_i \in \Theta_S^d$, the set of angles described via inequalities (E_1, E_2) in (13) satisfies,

$$E_1 \subset \Theta \quad ; \quad E_2 \subset \Theta \quad (16)$$

The key implication of (16) is that, all the requisite range measurements to completely verify if the associated ice-cone is safe are all available within the limited field-of-view.

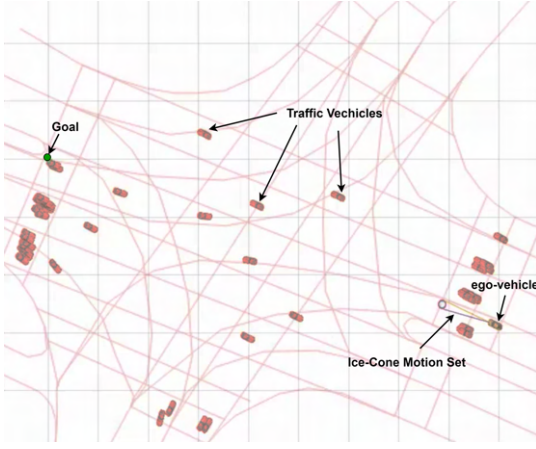


Fig. 4: Scenario 1 (Traffic intersection scenario from the interaction dataset)

B. Reference point selection

To select the reference point ($\mathcal{W}^k$) for describing the ice-cones for safe navigation, a distance-based metric to the navigation goal ($\mathcal{G} \in \mathbb{R}^2$) is chosen. During each iteration, the strategy for describing the reference point is as follows,

$$\mathcal{W}^k = \arg \min_X \|X - \mathcal{G}\| \quad \text{subject to, } X \in \mathcal{A} \quad (17)$$

The key idea behind the above strategy is that $\mathcal{W}^k$ for describing the ice-cone is chosen such that it is as close as possible to $\mathcal{G}$ within the safe admissible set $\mathcal{A}$ during each iteration. The ego-vehicle converges to $\mathcal{G}$ when the following is sufficiently true over all successive iterations,

$$\|\mathcal{W}^{k+1} - \mathcal{G}\| < \|\mathcal{W}^k - \mathcal{G}\|$$

The above condition is generally satisfied when set $\mathcal{A}$ progressively grows/moves towards $\mathcal{G}$ during subsequent iterations.

Remark 5: If such $\mathcal{W}^{k+1}$ is not available due to obstacles, then intermediate locations can be explored to ensure that eventually $\|\mathcal{W}^{k+1} - \mathcal{G}\| < \|\mathcal{W}^k - \mathcal{G}\|$

Remark 6: The objective function in (17) is a greedy value function for each $X \in \mathcal{A}$. In complex operating maps, learning-based algorithms (for example, value function-based reinforcement learning algorithms) can be potentially deployed here to determine $\mathcal{W}^k$ for intelligent navigation. This will be a subject of future work.

V. SIMULATION RESULTS

We examine three challenging scenarios from the perspective of handling dynamic entities in the environment through simulations. In the first scenario, we examine the safe navigation of ego-vehicle in a busy road junction where ego-vehicle does not follow any traffic rules. In the other two scenarios, we examine head-on collision possibilities.

A. Scenario 1 - Navigation in a busy road intersection

This scenario (illustrated in Figure 4) consists of moving agents whose trajectory information is obtained from the

interaction dataset [19]. The moving agents are illustrated as red cars, and the ego-vehicle is represented as the yellow car with an ice-cone motion set from the front axle. The tracks in the background are only used for representative purposes to illustrate the map (from the interaction dataset) over which the data is collected. The navigation goal is represented via a green marker.

In Figure 5, a safe navigation sequence via the ice-cone motion sets is illustrated. Since the proposed navigator is sensor-based, the LiDAR information for control computations is simulated from the traffic trajectory information from the dataset. As the robot navigates through its environment, it can be observed that the ice-cones are sequentially generated within the temporally varying space constraints (free-space) while simultaneously getting attracted to the navigation goal to ensure safe navigation.

B. Scenarios 2 & 3: Head-on collision avoidance

The ROS simulations are carried out on an HP Victus Gaming Laptop, which runs an i7 12th-gen processor, 16 GB RAM (DDR4), and has an NVidia GeForce- RTX 3050 graphics card. On average, the time taken for safe control computations via the proposed navigator was recorded to be approx. 8 ms during each iteration.

The cars deployed in this simulation are the CAT vehicle testbeds [20] that are equipped with front-facing LiDARs with a limited field of view. The LiDAR measurements consists of 120 data points, and they are updated at a frequency of 40 Hz. The point cloud information from the LiDARs is then used to determine the dynamically varying space constraints for safe control computations for navigation via ice-cone motion sets. The proposed ice-cone based navigator is illustrated via the following dynamic scenarios.

Video link: <https://youtu.be/LRf-sf4wj3I>

1) *Scenario 2:* A perpendicular crossing scenario in an urban neighborhood-type world is illustrated in Figure 6. In this, the cars would have to safely cross themselves to reach their respective navigation goals set at perpendicularly opposite ends of the neighborhood world. Here, both the cars are ego-vehicles that operate through the proposed ice-cone-based navigator (via local sensing). The associated navigation snapshots are illustrated in Figure 8.

2) *Scenario 3:* A parallel crossing scenario in a road-like environment is illustrated in Figure. 7. Here, the navigation goals are set towards the opposing ends of the road, and all three vehicles are ego-vehicles operating through the proposed ice-cone-based navigator (via local sensing). The cars would have to safely cross themselves to reach their respective navigation goals. The associated navigation snapshots are illustrated in Figure 9.

VI. CONCLUSIONS

This work addresses challenging scenarios for safe navigation of a car-like robot in the presence of multiple dynamic entities. The purpose is to avoid predicting the motion of dynamic obstacles and explicit requirement of adhering to paths/trajectories to ensure safety. A novel sensor-based

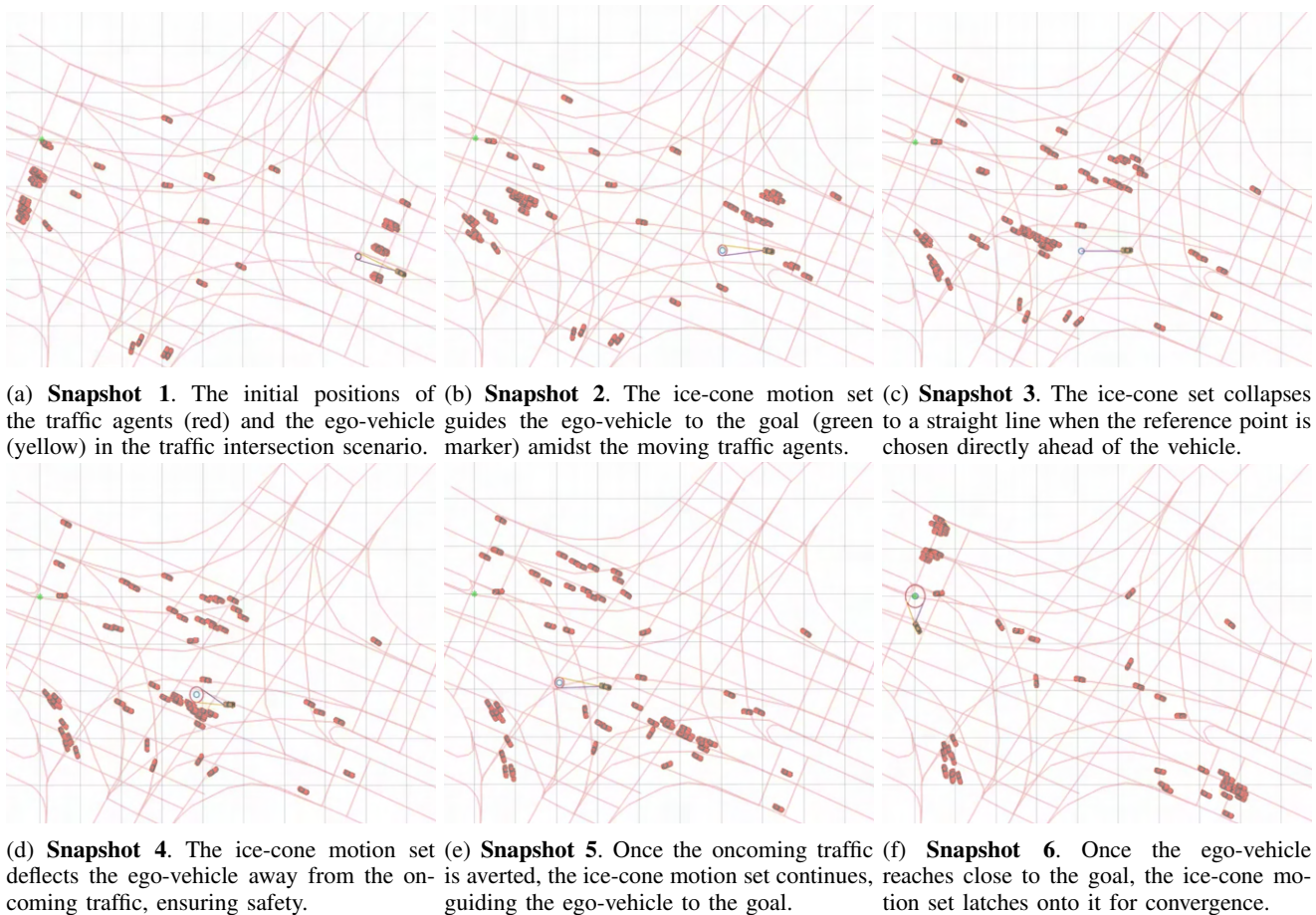


Fig. 5: Safe navigation in a traffic scenario via ice-cone motion sets of the feedback controller.

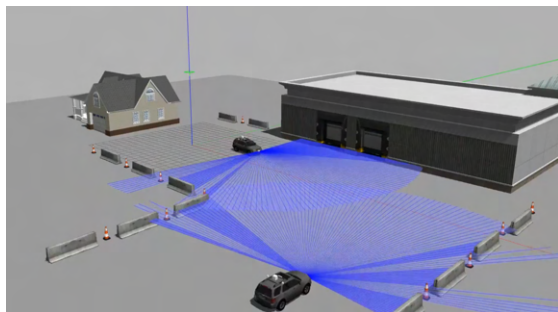


Fig. 6: Scenario 2 (Perpendicular crossing)

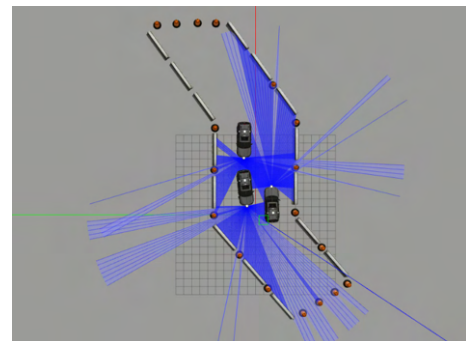


Fig. 7: Scenario 3 (Road-like environment)

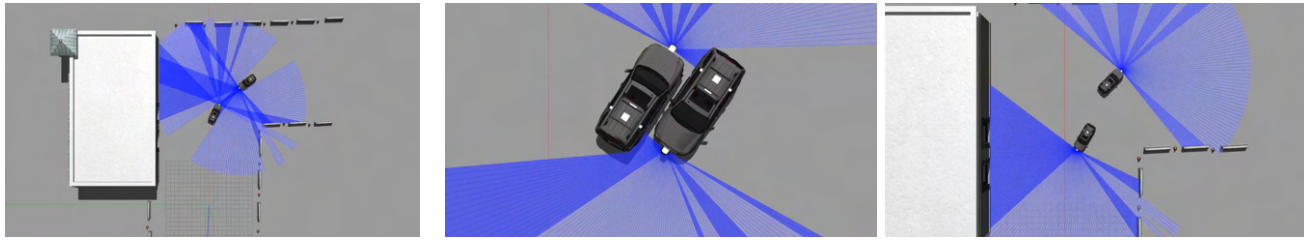
(with limited field-of-view) navigation strategy based on the motion sets (ice-cones) associated with feedback controllers for front wheel-driven car-like robots is presented in this work. The proposed strategy directly computes safe control inputs for navigation based on the point cloud information obtained during run-time.

While the proposed method has a potential for implementing in autonomous driving, its safe navigation is guaranteed when any other dynamic entity is not purposely planning to collide (similar to a road traffic scenario as against combat situations). Therefore, the presented strategy has the potential for applications in dense, crowded environments

where explicit prediction of neighboring agents to compute safe trajectories during run-time is not feasible. Given the novel approach, the future work involves bench-marking the performance of proposed technique and determining intelligent learning-based techniques to compute reference points for safe navigation in crowded, complex environments.

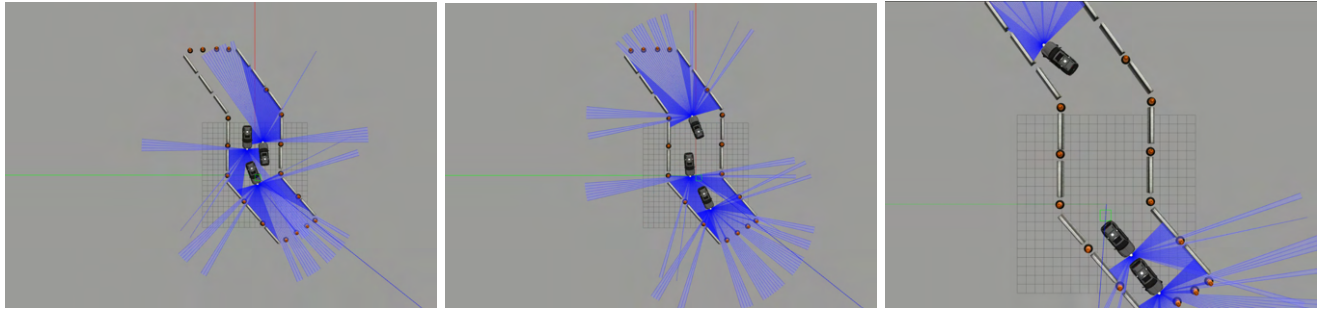
REFERENCES

- [1] James A. Douthwaite, Shiyu Zhao, and Lyudmila S. Mihaylova. Velocity obstacle approaches for multi-agent collision avoidance. *Unmanned Systems*, 07(01):55–64, 2019.



(a) **Snapshot 1.** The CAT vehicles approach each other to navigate to their goals. (b) **Snapshot 2.** The CAT vehicles safely pass each other. (c) **Snapshot 3.** The CAT vehicles then continue towards their navigation goal.

Fig. 8: Scenario 2 (Perpendicular Crossing)



(a) **Snapshot 1.** The CAT vehicles attempt to negotiate among themselves to ensure safety. (b) **Snapshot 2.** The CAT vehicles safely pass each other and approach their goals. (c) **Snapshot 3.** The CAT vehicles have reached their respective goals.

Fig. 9: Scenario 3 (Parallel crossing in road-like environment)

- [2] Javier Alonso-Mora, Andreas Breitenmoser, Martin Rufli, Paul Beardsley, and Roland Siegwart. *Optimal Reciprocal Collision Avoidance for Multiple Non-Holonomic Robots*, pages 203–216. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.
- [3] Jihao Huang, Jun Zeng, Xuemin Chi, Koushil Sreenath, Zhitao Liu, and Hongye Su. Velocity obstacle for polytopic collision avoidance for distributed multi-robot systems. *IEEE Robotics and Automation Letters*, 8(6):3502–3509, 2023.
- [4] Christoph Rösmann, Frank Hoffmann, and Torsten Bertram. Kinodynamic trajectory optimization and control for car-like robots. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5681–5686, 2017.
- [5] Sheng Zhu and Bilin Aksun-Guvenc. Trajectory planning of autonomous vehicles based on parameterized control optimization in dynamic on-road environments. *Journal of Intelligent & Robotic Systems*, 100(3):1055–1067, Dec 2020.
- [6] Ivo Batkovic, Ankit Gupta, Mario Zanon, and Paolo Falcone. Experimental validation of safe mpc for autonomous driving in uncertain environments. *IEEE Transactions on Control Systems Technology*, 31(5):2027–2042, 2023.
- [7] Mauro Salazar, Andrea Alessandretti, A. Pedro Aguiar, and Colin N. Jones. An energy efficient trajectory tracking controller for car-like vehicles using model predictive control. In *2015 54th IEEE Conference on Decision and Control (CDC)*, pages 3675–3680, Dec 2015.
- [8] Jun Zeng, Bike Zhang, and Koushil Sreenath. Safety-critical model predictive control with discrete-time control barrier function. In *2021 American Control Conference (ACC)*, pages 3882–3889, 2021.
- [9] Veronica Vulcano, Spyridon G. Tarantos, Paolo Ferrari, and Giuseppe Oriolo. Safe robot navigation in a crowd combining nmpe and control barrier functions. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 3321–3328, 2022.
- [10] Anoushka Alavilli, Khai Nguyen, Sam Schoedel, Brian Plancher, and Zachary Manchester. TinyMPC: Model-Predictive Control on Resource-Constrained Microcontrollers. *arXiv e-prints*, page arXiv:2310.16985, October 2023.
- [11] Shathushan Sivashangaran and Minghui Zheng. Intelligent autonomous navigation of car-like unmanned ground vehicle via deep reinforcement learning. *IFAC-PapersOnLine*, 54(20):218–225, 2021. Modeling, Estimation and Control Conference MECC 2021.
- [12] Andrew K. Mackay, Luis Riazuelo, and Luis Montano. RL-dovs: Reinforcement learning for autonomous robot navigation in dynamic environments. *Sensors*, 22(10), 2022.
- [13] Xiao Wang. Ensuring safety of learning-based motion planners using control barrier functions. *IEEE Robotics and Automation Letters*, 7(2):4773–4780, 2022.
- [14] Zhong Cao, Shaobing Xu, Xinyu Jiao, Huei Peng, and Diange Yang. Trustworthy safety improvement for autonomous driving using reinforcement learning. *Transportation Research Part C: Emerging Technologies*, 138:103656, 2022.
- [15] Shangding Gu, Long Yang, Yali Du, Guang Chen, Florian Walter, Jun Wang, Yaodong Yang, and Alois Knoll. A Review of Safe Reinforcement Learning: Methods, Theory and Applications. *arXiv e-prints*, page arXiv:2205.10330, May 2022.
- [16] A. De Luca, G. Oriolo, and C. Samson. *Feedback control of a non-holonomic car-like robot*, pages 171–253. Springer Berlin Heidelberg, Berlin, Heidelberg, 1998.
- [17] Veejay Karthik J and Leena Vachhani. Self-navigation in crowds: An invariant set-based approach. *arXiv e-prints*, page arXiv:2401.09375, January 2024.
- [18] Aykut İşleyen, Nathan van de Wouw, and Ömür Arslan. Feedback Motion Prediction for Safe Unicycle Robot Navigation. *arXiv e-prints*, page arXiv:2209.12648, September 2022.
- [19] Wei Zhan, Liting Sun, Di Wang, Haojie Shi, Aubrey Clausse, Maximilian Naumann, Julius Kümmerle, Hendrik Königshof, Christoph Stiller, Arnaud de La Fortelle, and Masayoshi Tomizuka. INTERACTION Dataset: An International, Adversarial and Cooperative moTION Dataset in Interactive Driving Scenarios with Semantic Maps. *arXiv:1910.03088 [cs, eess]*, 2019.
- [20] Rahul Bhadani, Jonathan Sprinkle, and Matthew Bunting. The CAT Vehicle Testbed: A Simulator with Hardware in the Loop for Autonomous Vehicle Applications. *Proceedings of 2nd International Workshop on Safe Control of Autonomous Vehicles (SCAV 2018)*, Porto, Portugal, 10th April 2018, *Electronic Proceedings in Theoretical Computer Science* 269, pp. 32–47, 2018.